Game Design Document: "Syntax"

> **Mission Statement:** An un-themed, utilitarian mobile browser game designed to combat developer skill atrophy. In an era of LLM-generated code, "Syntax" tests and reinforces raw logic, syntax memory, and algorithmic understanding using touch-native, daily micro-challenges.

1. Core Gameplay Loop

The game relies on habit formation through a synchronized daily reset, completely independent of complex fictional narratives. The player's motivation is self-improvement and peer competition.

- **Login:** User opens the browser app. No heavy loads, instant access.
- **The Daily Stack:** Player is presented with 3 to 5 micro-challenges for the day.
- **Execution:** Player solves the challenges using tap, drag-and-drop, and swipe mechanics (zero manual typing required).
- **Resolution:** The daily score is generated based on accuracy and time.
- **Progression:** Experience points (XP) are allocated to specific language skill trees based on the day's challenge stack.

2. Game Mechanics & Question Styles

Challenges are designed to test real programming knowledge without the friction of mobile keyboards. The system is modular, allowing for continuous expansion of question types.

Current Scope (Launch Set)

- **Bug Spotting:** A 10-15 line snippet is displayed. The player must tap the exact line containing a logic error, memory leak, or syntax fault.
- **Parsons Problems:** Code blocks are provided in a scrambled list. The player drags and drops them into the correct logical order and indentation level (e.g., assembling a Binary Search or a custom array sort).
- **The LLM Auditor:** A prompt and its AI-generated code response are shown. The player selects from multiple-choice options detailing the specific edge case the AI failed to account for.
- **Time Complexity Match:** An algorithm is shown, and the user must assign its Big O notation (time or space) from a pre-defined set of buttons.

Future Expansion Scope

- **Architecture Assembly:** Dragging and dropping cloud/infrastructure components (Load Balancers, DBs, Caches) to satisfy a high-level system design requirement.

- **Regex Crosswords:** A visual grid where rows and columns are defined by regular expressions, testing pattern matching skills.
- **State Prediction:** Showing a snippet with a complex state change (e.g., nested loops or asynchronous callbacks) and asking the user to input the final value of a specific variable.

3. Meta-Game: Skill Trees & Progression

Instead of arbitrary levels, the progression system mirrors professional development. Players earn nodes on specific language and framework trees.

- **Base Trees:** The game will initially support distinct tracks. For example, a player might focus their profile on *Python*, *JavaScript*, or *Flutter/Dart*.
- **Nodes & Unlocks:** Accumulating XP in Python might unlock challenges involving specific advanced libraries (e.g., Pandas logic) or deeper language features (e.g., metaclasses, generators).
- **Proficiency Badge:** A player's visual rank is directly tied to their skill tree density, serving as a realistic indicator of their daily retention efforts.

4. Social & Competitive (Wordle Mechanics)

To drive daily retention without requiring traditional multiplayer infrastructure, the game utilizes asynchronous, standardized sharing.

> **Universal Daily Seed:** Every player receives the exact same "Daily Stack" at midnight local time. The shared struggle is the community hook.

Shareable Results

Upon completing the Daily Stack, players can copy an emoji grid to the clipboard. The grid obfuscates the answers while bragging about performance. Colors represent first-try success, multiple attempts, or failure.

```
Syntax Daily #142
Python Track: Lvl 12
⏱️ 02:14
1️⃣ 🟩 (Spot the Bug)
2️⃣ 🟩 (Parsons Problem)
3️⃣ 🟨 (LLM Audit)
```

5. Visual & Audio Style

- **Aesthetic:** Strict "Dark Mode IDE" design language. Utilitarian, clean, with high-contrast syntax highlighting matching popular editors like VS Code.
- **UI Framework:** Minimalist interfaces. Large touch targets for drag-and-drop. Haptic feedback on mobile (vibrations on snapping a block into place).
- **Audio:** No background music. Subtle, mechanical sound effects—like mechanical keyboard clicks when a correct answer is locked in.